

Universidad Tecnológica de Puebla

Tecnologías de la Información, área Desarrollo de Software Multiplataforma

Materia: Desarrollo de Aplicaciones Web Progresivas

GUÍA DE TRABAJO EN EQUIPO

Subir el proyecto a GitHub

Proyecto: AlertaVecinal, red de seguridad vecinal (PWA)

Profesora: [NOMBRE_PROFESORA]

Grupo: 10 D

Integrantes: Serrano Arana Jahir Jaziel · Matrícula [MATRICULA_1]
Xicali Diaz Luis Alberto · Matrícula [MATRICULA_2]
Gonzalez Barrios Yeudiel Nain · Matrícula [MATRICULA_3]

Fecha: 29 de octubre de 2026

Índice

1. Reparto de archivos **3**

Integrante 1: Serrano Arana Jahir Jaziel 3

Integrante 2: Xicali Diaz Luis Alberto 4

Integrante 3: Gonzalez Barrios Yeudiel Nain 4

Archivos que NO se suben 4

¿Y los archivos que necesitan dos personas? 5

2. Instalar Git y configurar tu nombre (cada integrante, en su computadora) **5**

2.1 Instalar Git 5

2.2 Configurar nombre y correo 6

3. Integrante 1: crear el repositorio y subir los commits 1 y 2 ..... **7**

3.1 Crear el repositorio en GitHub 7

3.2 Agregar a los compañeros como colaboradores 7

3.3 Clonar y preparar las variables 7

3.4 Commit 1: Aplicación web base 8

3.5 Commit 2: Conversión a PWA 8

4. Orden obligatorio de los turnos **8**

5. Integrantes 2 y 3: clonar, copiar y subir **9**

5.1 Clonar (una sola vez) 9

5.2 Antes de cada commit: traer lo último 9

5.3 Comandos de cada commit 9

6. Siempre git pull antes de push, y qué hacer si hay un conflicto **12**

7. Verificar que aparecen los 3 integrantes **13**

8. Recordatorios importantes **13**

Esta guía explica, paso a paso y con comandos para **Windows (PowerShell)**, cómo los tres integrantes suben el proyecto a un mismo repositorio de GitHub, cada uno con **sus propios commits**, para que GitHub registre la autoría de cada quien.

Integrante	Nombre	Responsabilidad
Integrante 1	Serrano Arana Jahir Jaziel	Base web, caché y aportación "Alerta sin señal"
Integrante 2	Xicali Diaz Luis Alberto	Mapa, notificaciones push y aportación "Mapa de calor"
Integrante 3	Gonzalez Barrios Yeudiel Nain	Panel de administración, instalación y aportación "Vecino Verificado Pro"

> **Carpeta del proyecto completo:** es la carpeta que contiene todo el proyecto ya terminado (por ejemplo `C:\Users\TU_USUARIO\Desktop\PWA`). Cada integrante necesita una copia de ella (USB, Google Drive, etc.). De ahí **copia solo sus archivos** al repositorio.

---

1. Reparto de archivos

Cada archivo del proyecto queda asignado a **un solo integrante**.

Integrante 1: Serrano Arana Jahir Jaziel

Commit	Mensaje	Archivos
1	`Aplicación web base`	`index.html`, `css/estilos.css`, `js/firebase-config.js`, `js/auth.js`, `js/alertas.js`, `firebase.json`, `firestore.rules`, `.gitignore`
2	`Conversión a PWA: manifest y caché`	`manifest.json`, `iconos/` (4 PNG), `sw.js`, `offline.html`, `demo-cache.html`, `js/cache-demo.js`, `js/app.js`
3	`Aportación: Alerta sin señal`	`js/offline-sync.js`, `sw-sync.js`
4	`Documentación Subproducto 4`	`entregables/Subproducto4_API_Cache.pdf`

Integrante 2: Xicali Diaz Luis Alberto

Commit	Mensaje	Archivos
1	`Mapa de alertas con Leaflet y Nominatim`	`js/mapa.js`
2	`Notificaciones push con FCM`	`sw-push.js`, `js/push.js`, `demo-push.html`, `scripts/enviar-push.js`, `scripts/package.json`
3	`Aportación: Mapa de calor de incidentes`	`js/heatmap.js`
4	`Documentación Subproducto 6`	`entregables/Subproducto6_Notificaciones.pdf`

Integrante 3: Gonzalez Barrios Yeudiel Nain

Commit	Mensaje	Archivos
1	`Panel de administración de usuarios`	`js/admin.js`, `admin.html`
2	`Botón de instalación de la PWA`	`js/instalar.js`
3	`Aportación: Vecino Verificado Pro`	`estadisticas.html`, `js/estadisticas.js`
4	`Documentación Subproducto 5 y README`	`entregables/Subproducto5_Aportacion.pdf`, `README.md`, `entregables/GUIA_REPOSITORIO.md`, `entregables/GUIA_REPOSITORIO.pdf`

Archivos que NO se suben

Archivo o carpeta	Motivo
`scripts/serviceAccountKey.json`	Clave privada de Firebase. **Nunca** se sube (ya está en `.gitignore`).
`scripts/node_modules/`	Se instala con `npm install`.
`reportes/`	Herramientas locales para probar la app y generar los PDF.

Archivo o carpeta	Motivo
`.firebaserc`, `.firebase/`	Los crea Firebase CLI en la computadora de quien hace el deploy.

> Si `firebase init` creó el archivo `firestore.indexes.json`, el Integrante 1 lo sube junto con su commit 4.

¿Y los archivos que necesitan dos personas?

El proyecto se diseñó para que **nadie tenga que editar archivos de otro integrante**:

- `index.html` (Integrante 1) ya carga todos los módulos con un cargador que **ignora los que aún no existen**. Cada módulo agrega por sí mismo su enlace del menú y su tarjeta en el panel. Por eso no hay que agregar líneas a `index.html`.
- `sw.js` (Integrante 1) ya incluye `importScripts('sw-push.js')` y `importScripts('sw-sync.js')` dentro de `try/catch`: funciona aunque esos archivos todavía no se hayan subido.
- `css/estilos.css` (Integrante 1) ya trae los estilos de todas las pantallas.

Resultado: **en cada commit intermedio la app abre y funciona**. Antes de un archivo faltante solo aparece un aviso en la consola del navegador, no un error.

---

2. Instalar Git y configurar tu nombre (cada integrante, en su computadora)

2.1 Instalar Git

Abre **PowerShell** (tecla Windows, escribe `PowerShell`, Enter) y ejecuta:

```
winget install --id Git.Git -e --source winget
```

Si `winget` no existe, descarga el instalador de https://git-scm.com/download/win y acepta las opciones por defecto. **Cierra y vuelve a abrir PowerShell** y comprueba:

```
git --version
```

2.2 Configurar nombre y correo

El correo **debe ser el mismo de tu cuenta de GitHub** (GitHub > foto > *Settings* > *Emails*). Si ahí está activada la opción *Keep my email addresses private*, usa el correo que termina en `@users.noreply.github.com` que aparece en esa página.

Integrante 1:

```
git config --global user.name "Serrano Arana Jahir Jaziel"
git config --global user.email "CORREO_DE_GITHUB_DEL_INTEGRANTE_1"
```

Integrante 2:

```
git config --global user.name "Xicali Diaz Luis Alberto"
git config --global user.email "CORREO_DE_GITHUB_DEL_INTEGRANTE_2"
```

Integrante 3:

```
git config --global user.name "Gonzalez Barrios Yeudiel Nain"
git config --global user.email "CORREO_DE_GITHUB_DEL_INTEGRANTE_3"
```

Todos, para que `git pull` no haga preguntas:

```
git config --global pull.rebase false
git config --global init.defaultBranch main
```

Comprueba que quedó bien:

```
git config --global --list
```

> **¿Comparten una misma computadora?** Entonces **no** usen `--global` : dentro de la carpeta del repositorio, cada integrante ejecuta `git config user.name "..."` y `git config user.email "..."` **justo antes de su commit**. Al hacer `push` , si Windows pide iniciar sesión en GitHub, cada quien entra con su cuenta.

---

3. Integrante 1: crear el repositorio y subir los commits 1 y 2

3.1 Crear el repositorio en GitHub

1. Entra a https://github.com/new.
2. *Repository name*: `alertavecinal`. Elige **Public** (o *Private* si la profesora tendrá acceso).
3. **No marques** *Add a README*, *.gitignore* ni *license* (el repositorio debe nacer vacío).
4. Pulsa **Create repository** y copia la dirección `https://github.com/TU_USUARIO/alertavecinal.git`.

3.2 Agregar a los compañeros como colaboradores

1. En el repositorio: **Settings** > **Collaborators** (en *Access*) > **Add people**.
2. Escribe el usuario de GitHub del Integrante 2 > **Add**. Repite con el Integrante 3.
3. Cada compañero recibe un correo: debe abrirlo y pulsar **Accept invitation** (o entrar a `https://github.com/TU_USUARIO/alertavecinal/invitations`).

3.3 Clonar y preparar las variables

En PowerShell (cambia `TU_USUARIO` y la ruta de la carpeta del proyecto completo):

```
cd $HOME\Desktop
git clone https://github.com/TU_USUARIO/alertavecinal.git
cd alertavecinal
$origen = "C:\Users\TU_USUARIO_DE_WINDOWS\Desktop\PWA"
```

> Git avisará *"You appear to have cloned an empty repository"*: es normal. La variable `$origen` se pierde si cierras PowerShell; vuelve a escribirla si es necesario.

3.4 Commit 1: Aplicación web base

```
New-Item -ItemType Directory -Force css, js | Out-Null
Copy-Item "$origen\index.html", "$origen\firebase.json", "$origen\firestore.rules",
"$origen\.gitignore" .
Copy-Item "$origen\css\estilos.css" css\
Copy-Item "$origen\js\firebase-config.js", "$origen\js\auth.js", "$origen\js\alertas.js" js\
git add .
git status
git commit -m "Aplicación web base"
git branch -M main
git push -u origin main
```

`git status` debe mostrar solo los 8 archivos del commit 1.

3.5 Commit 2: Conversión a PWA

```
Copy-Item "$origen\manifest.json", "$origen\sw.js", "$origen\offline.html", "$origen\demo-
cache.html" .
Copy-Item "$origen\iconos" . -Recurse
Copy-Item "$origen\js\cache-demo.js", "$origen\js\app.js" js\
git add .
git commit -m "Conversión a PWA: manifest y caché"
git pull
git push
```

Avisa al Integrante 2 que ya puede empezar.

---

4. Orden obligatorio de los turnos

Para que la app funcione después de cada paso, respeten este orden:

Turno	Quién	Commits
1	Integrante 1	Commit 1 y Commit 2
2	Integrante 2	Commit 1 (mapa)
3	Integrante 3	Commit 1 (administración) y Commit 2 (instalación)
4	Integrante 2	Commit 2 (push)

Turno	Quién	Commits
5	Los tres, en cualquier orden	Integrante 1: commits 3 y 4 · Integrante 2: commits 3 y 4 · Integrante 3: commits 3 y 4

> Los commits de documentación (PDF) se hacen **después** de pegar la URL de la app en `reportes/datos.json` y regenerar los PDF (ver README, sección 9).

---

5. Integrantes 2 y 3: clonar, copiar y subir

5.1 Clonar (una sola vez)

```
cd $HOME\Desktop
git clone https://github.com/USUARIO_DEL_INTEGRANTE_1/alertavecinal.git
cd alertavecinal
$origen = "C:\Users\TU_USUARIO_DE_WINDOWS\Desktop\PWA"
```

5.2 Antes de cada commit: traer lo último

```
git pull
```

5.3 Comandos de cada commit

Integrante 2, Commit 1: Mapa de alertas con Leaflet y Nominatim

```
git pull
Copy-Item "$origen\js\mapa.js" js\
git add .
git commit -m "Mapa de alertas con Leaflet y Nominatim"
git pull
git push
```

Integrante 3, Commit 1: Panel de administración de usuarios

```
git pull
Copy-Item "$origen\js\admin.js" js\
Copy-Item "$origen\admin.html" .
git add .
git commit -m "Panel de administración de usuarios"
git pull
git push
```

Integrante 3, Commit 2: Botón de instalación de la PWA

```
git pull
Copy-Item "$origen\js\instalar.js" js\
git add .
git commit -m "Botón de instalación de la PWA"
git pull
git push
```

Integrante 2, Commit 2: Notificaciones push con FCM

```
git pull
New-Item -ItemType Directory -Force scripts | Out-Null
Copy-Item "$origen\sw-push.js", "$origen\demo-push.html" .
Copy-Item "$origen\js\push.js" js\
Copy-Item "$origen\scripts\enviar-push.js", "$origen\scripts\package.json" scripts\
git add .
git status
git commit -m "Notificaciones push con FCM"
git pull
git push
```

Revisa en `git status` que **no** aparezca `serviceAccountKey.json`.

Integrante 1, Commit 3: Aportación: Alerta sin señal

```
git pull
Copy-Item "$origen\js\offline-sync.js" js\
Copy-Item "$origen\sw-sync.js" .
git add .
git commit -m "Aportación: Alerta sin señal"
git pull
git push
```

Integrante 1, Commit 4: Documentación Subproducto 4

```
git pull
New-Item -ItemType Directory -Force entregables | Out-Null
Copy-Item "$origen\entregables\Subproducto4_API_Cache.pdf" entregables\
git add .
git commit -m "Documentación Subproducto 4"
git pull
git push
```

Integrante 2, Commit 3: Aportación: Mapa de calor de incidentes

```
git pull
Copy-Item "$origen\js\heatmap.js" js\
git add .
git commit -m "Aportación: Mapa de calor de incidentes"
git pull
git push
```

Integrante 2, Commit 4: Documentación Subproducto 6

```
git pull
New-Item -ItemType Directory -Force entregables | Out-Null
Copy-Item "$origen\entregables\Subproducto6_Notificaciones.pdf" entregables\
git add .
git commit -m "Documentación Subproducto 6"
git pull
git push
```

Integrante 3, Commit 3: Aportación: Vecino Verificado Pro

```
git pull
Copy-Item "$origen\estadisticas.html" .
Copy-Item "$origen\js\estadisticas.js" js\
git add .
git commit -m "Aportación: Vecino Verificado Pro"
git pull
git push
```

Integrante 3, Commit 4: Documentación Subproducto 5 y README

```
git pull
New-Item -ItemType Directory -Force entregables | Out-Null
Copy-Item "$origen\README.md" .
Copy-Item "$origen\entregables\Subproducto5_Aportacion.pdf",
"$origen\entregables\GUIA_REPOSITORIO.md", "$origen\entregables\GUIA_REPOSITORIO.pdf"
entregables\
git add .
git commit -m "Documentación Subproducto 5 y README"
git pull
git push
```

---

6. Siempre git pull antes de push, y qué hacer si hay un conflicto

Regla: antes de cada `git push`, ejecuta `git pull`. Así traes los commits que tus compañeros subieron mientras trabajabas.

Mensaje	Qué significa	Qué hacer
`! [rejected] main -> main (fetch first)`	Alguien subió algo antes que tú.	`git pull` y luego `git push`.
Se abre un editor con "Merge branch 'main'..."	Git creó un commit de unión.	En el editor *Vim*: escribe `:wq` y Enter. En el Bloc de notas: guarda y cierra.
`CONFLICT (content): Merge conflict in ARCHIVO`	Dos personas cambiaron las mismas líneas.	Sigue los pasos de abajo.

Resolver un conflicto (con este reparto no debería pasar, porque nadie toca archivos de otro):

1. Ejecuta `git status`: los archivos en conflicto aparecen como *both modified*.
2. Abre cada archivo con un editor (por ejemplo VS Code). Verás bloques así:

```
<<<<<<< HEAD
tu versión
=======
la versión de tu compañero
>>>>>>> origin/main
```

3. Deja el texto correcto (normalmente **ambas** partes) y borra las tres líneas marcadoras `<<<<<<<`, `=======` y `>>>>>>>`.
4. Guarda y ejecuta:

```
git add .
git commit -m "Resolver conflicto"
git push
```

5. Si te equivocaste a mitad del proceso y quieres volver a empezar: `git merge --abort` y repite `git pull`.

---

7. Verificar que aparecen los 3 integrantes

1. En GitHub abre el repositorio > pestaña **Insights** > **Contributors**. Deben aparecer las tres cuentas con sus commits. (La gráfica puede tardar unos minutos en actualizarse.)
2. También puedes revisar **Commits** (en la página principal, el reloj con el número de commits): cada commit muestra la foto y el nombre de su autor.
3. Desde PowerShell:

```
git log --pretty=format:"%h  %an  <%ae>  %s"
```

4. Si un commit aparece con un nombre gris y sin foto, el correo de `git config` no coincide con la cuenta de GitHub: agrega ese correo en GitHub > *Settings* > *Emails* (o corrige `user.email` para los siguientes commits).

---

8. Recordatorios importantes

- **Nunca subas `serviceAccountKey.json`.** Da acceso total al proyecto de Firebase. Ya está en `.gitignore`; aun así, revisa `git status` antes de cada commit.
- Si por error se subió, en la consola de Firebase > *Cuentas de servicio* **elimina esa clave y genera otra**: borrarla del repositorio no basta, porque queda en el historial.
- No uses `git add` con archivos de otro integrante: cada quien sube solo lo suyo.
- No inicialices otro repositorio (`git init`) dentro de la carpeta clonada.

- Después de cada commit, cualquiera puede probar la app con `npx serve -l 3000` y abrir http://localhost:3000/?demo=1.